

UniTrack NFC

Backend Developer

Documentation

ESP32 + PN532 NFC Hardware · REST API · Database Schema
Business Logic · Security · Deployment Guide

Document Type	Backend Developer Reference
System	ESP32 + PN532 NFC Attendance
Version	v2.0
Base URL	<code>http://<server>:8000/api</code>
Modes	Entry · Exit · Room · Admin
Status	Active / Production Ready

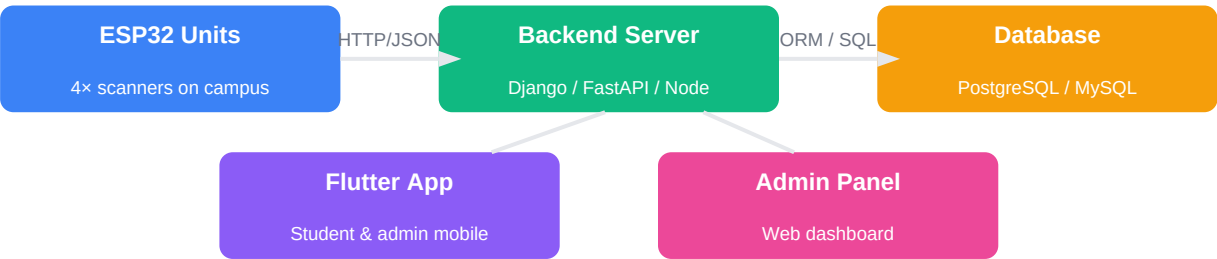
Table of Contents

1	System Architecture	...
2	Authentication & Security	...
3	Operating Modes Overview	...
4	Common Request / Response Schema	...
5	Endpoint: POST /attendance/entry Entry Gate	...
6	Endpoint: POST /attendance/exit Exit Gate	...
7	Endpoint: POST /attendance/room Room / Lab	...
8	Endpoint: POST /admin/card Admin Panel	...
9	HTTP Status Code Reference	...
10	Business Logic Rules	...
11	Database Schema	...
12	Environment Configuration	...
13	NVS Device Provisioning	...
14	Security Hardening Guide	...
15	Deployment Checklist	...

1 System Architecture

The UniTrack NFC system connects physical ESP32 scanner units on campus to a central backend server shared with the university Flutter mobile application. Each ESP32 operates in one of four modes and POSTs a JSON payload to the backend every time a student NFC card is scanned. The backend validates the card, applies mode-specific attendance logic, and returns a structured response that drives the scanner's LED and buzzer feedback.

Component Overview



Data Flow — Card Scan to Response

Step	Actor	Action
1	Student	Presents NFC card to ESP32 scanner
2	ESP32	PN532 reads UID → builds JSON payload → POST to endpoint
3	Backend	Validates shared secret → rejects if mismatch (401)
4	Backend	Looks up card UID in cards table → 404 if unknown
5	Backend	Applies mode-specific business logic (entry session, room schedule...)
6	Backend	Writes attendance record to database
7	Backend	Returns JSON { allowed, message, sub }
8	ESP32	Drives LED + buzzer; displays message on OLED for 1.5 s

2 Authentication & Security

There is no OAuth or JWT between the ESP32 and backend. Authentication uses a **shared secret** — a pre-configured string sent in every request body. This is appropriate for a trusted LAN environment. When deploying over the internet, combine with HTTPS (TLS) as described in Section 14.

Shared Secret Validation

Every request from the ESP32 includes a `secret` field. The backend compares it against the `ESP32_SECRET` environment variable. Any mismatch must return **HTTP 401** immediately — before any database query.

Environment variable + validation:

```
# .env
ESP32_SECRET=your-strong-random-secret-here

# Validation pseudocode (runs first in every endpoint handler)
if request.body["secret"] != os.environ["ESP32_SECRET"]:
```

```
return HTTP 401 { "error": "Invalid secret" }
```

Device Identity

Every request also includes a `device_id` field — the physical unit's name (e.g. "gate-north", "lab-301"). Store this in all attendance and audit records. It lets administrators trace every scan back to the exact scanner that processed it.

Never expose `ESP32_SECRET` in source code, logs, or API responses.
Store it only in environment variables or a secrets manager.

3 Operating Modes Overview

Each physical ESP32 unit operates in exactly one mode at a time. The mode is switched by a 2-second capacitive touch long-press and persisted in NVS flash. The mode determines which endpoint is called and which JSON fields are sent.

0 ENTRY GATE [/attendance/entry](#)

Records student arrival on campus. Creates an open campus session. This session is the prerequisite for all ROOM scans — a student cannot register for a lecture without first scanning at the entry gate.

1 EXIT GATE [/attendance/exit](#)

Records student departure. Closes the open entry session and stores on-campus duration. Also sends `exit_time` to the mobile app for display.

2 ROOM / LAB [/attendance/room](#)

Checks a student into a lecture hall or lab. Enforces the gate-first rule by returning HTTP 403 if no open entry session exists for today. Requires `room_id` to be pre-configured on the device via NVS.

3 ADMIN PANEL [/admin/card](#)

Card management: enroll new cards, deactivate lost/stolen cards, or query card info. The action (enroll / deactivate / info) is pre-configured on each admin unit via the NVS key `admin_action`.

Common Request / Response Schema

Common Request Fields (sent in every POST body)

Field	Type	Required	Description
uid	String	Yes	NFC card UID, colon-separated uppercase hex. E.g. "A1:B2:C3:D4". Validate: ^([0-9A-F]{2}:)*[0-9A-F]{2}\$
secret	String	Yes	Shared secret — compared against ESP32_SECRET env var. Reject with 401 on mismatch
device_id	String	Yes	Physical unit name. E.g. "gate-north", "lab-301". Store in all attendance records.

Common Response Fields (returned in every 200 OK body)

The ESP32 firmware reads exactly these three fields from every 200 response. Keep message short (≤ 13 chars) — it renders on a 0.96" OLED.

Field	Type	Required	Description
allowed	Boolean	Yes	true = grant access / play success feedback. false = deny / play denied feedback.
message	String	Yes	Primary OLED line. Max 13 chars. Examples: "WELCOME", "REGISTERED", "ACCESS D
sub	String	No	Optional second OLED line. Max 21 chars. Examples: "Ahmed Hassan", "Networks Lec 3"

Feedback Behaviour Driven by HTTP Code + allowed Field

HTTP Code	allowed	LED	Buzzer	OLED
200	true	Green flash	1 × 1 kHz 150 ms	message + sub
200	false	Red × 2	2 × low 400 Hz	message + sub
200	—	Blue flash	Ascending 2-tone	message + sub (admin mode)
401	—	Red solid	Long low 250 Hz 500 ms	"SERVER ERROR"
403	—	Red × 2	2 × low 400 Hz	"ACCESS DENIED" / "Gate login first"
404	—	Red × 2	2 × low 400 Hz	"UNKNOWN CARD" / "Not enrolled"
409	—	Red × 2	2 × low 400 Hz	"ALREADY" / "Registered"
500	—	Red solid	Long low 250 Hz 500 ms	"SERVER ERROR"

5 POST /attendance/entry — Entry Gate

Records a student arriving on campus. Creates an open campus session record. This is the prerequisite for all Room/Lab scans — students must scan at the entry gate before they can register for any lecture or lab session.

POST /attendance/entry

Example request body

```
{
  "uid": "A1:B2:C3:D4",
  "secret": "your-strong-secret",
  "device_id": "gate-north"
}
```

Server-side logic

1. Validate secret against ESP32_SECRET. Return 401 on mismatch.
2. Look up student by uid in the cards table. Return 404 if not enrolled.
3. Check campus_sessions for an open session today (entry_time IS NOT NULL AND exit_time IS NULL). Return 409 if found.
4. Insert new campus session: { student_id, device_id, entry_time=NOW(), exit_time=NULL }
5. Return 200 with the student's name in sub.

Response examples

Success (200):

```
{
  "allowed": true,
  "message": "WELCOME",
  "sub": "Ahmed Hassan"
}
```

Error responses

Code	Condition	ESP32 Displays
401	Secret does not match ESP32_SECRET	SERVER ERROR
404	UID not found in cards table	UNKNOWN CARD / Not enrolled
409	Student already has an open entry session today	ALREADY / Registered

6 POST /attendance/exit — Exit Gate

Records a student departing campus. Finds and closes the open entry session created by /attendance/entry, storing the exit time and computing on-campus duration. The Flutter app uses this data to display attendance summaries.

POST /attendance/exit

Example request body

```
{
  "uid": "A1:B2:C3:D4",
```

```
"secret": "your-strong-secret",
"device_id": "gate-south"
}
```

Server-side logic

- 1. Validate secret. Return 401 on mismatch.
- 2. Look up student by uid. Return 404 if not enrolled.
- 3. Find the most recent open entry session for today (exit_time IS NULL). If none found, log the anomaly but still return 200 — the student may have entered before the system was installed.
- 4. Update session: exit_time = NOW(). Duration is computed as exit_time - entry_time.
- 5. Return 200.

Response examples

Success (200):

```
{
  "allowed": true,
  "message": "GOODBYE",
  "sub": "Ahmed Hassan"
}
```

Error responses

Code	Condition	ESP32 Displays
401	Secret does not match ESP32_SECRET	SERVER ERROR
404	UID not found in cards table	UNKNOWN CARD / Not enrolled

7 POST /attendance/room — Room / Lab

Records a student checking in to a lecture hall or laboratory. This endpoint enforces the core attendance rule: a student must have an open campus entry session to register for any room session. Returns HTTP 403 if the gate check fails.

POST /attendance/room

Mode-specific request fields

Field	Type	Required	Description
room_id	Integer	Yes	ID of the lecture room / lab. Pre-configured on the device via NVS key "room_id". Used to l

Example request body

```
{
  "uid": "A1:B2:C3:D4",
  "secret": "your-strong-secret",
  "device_id": "hall-b2",
  "room_id": 12
}
```

Server-side logic

- 1. Validate secret. Return 401 on mismatch.

- 2. Look up student by uid. Return 404 if not enrolled.
- 3. **Gate check:** query campus_sessions for an open entry session today (entry_time IS NOT NULL AND exit_time IS NULL). Return **403** if none found.
- 4. Look up the active schedule entry for room_id — within a ±15 min window or as per your university policy.
- 5. Check for duplicate: query room_attendance for this student + room_id + today. Return 409 if already registered.
- 6. Insert: { student_id, room_id, device_id, scanned_at=NOW() }
- 7. Return 200 with the lecture/course name in sub.

■ HTTP 403 is the core enforcement of the attendance policy.

The ESP32 shows "ACCESS DENIED / Gate login first" and plays the denied pattern.

Do not return 200 with allowed:false — use 403 so the ESP32 shows the right message.

Response examples

Success (200):

```
{
  "allowed": true,
  "message": "REGISTERED",
  "sub": "Networks Lec 3"
}
```

Student not on campus (403):

```
{
  "error": "Student has no open campus entry session today"
}

// ESP32 displays: ACCESS DENIED / Gate login first
// LED: Red x2 Buzzer: double low beep
```

Error responses

Code	Condition	ESP32 Displays
401	Secret does not match ESP32_SECRET	SERVER ERROR
403	No open campus entry session found for today	ACCESS DENIED / Gate login first
404	UID not found in cards table	UNKNOWN CARD / Not enrolled
409	Student already registered for this room today	ALREADY / Registered

8 POST /admin/card — Admin Panel

Card management endpoint used by admin units. The action field controls what the server does. It is pre-configured on each admin ESP32 unit via NVS key "admin_action". One unit can be provisioned for enrollment, another for deactivation.

POST /admin/card

Mode-specific request fields

Field	Type	Required	Description
action	String	Yes	"enroll" — link UID to a student account. "deactivate" — mark card as inactive (lost/stolen). "info" — return card owner + status for display on OLED.

Example request body

```
{
  "uid": "A1:B2:C3:D4",
  "secret": "your-strong-secret",
  "device_id": "admin-office",
  "action": "enroll"
}
```

Server-side logic

- 1. Validate secret. Return 401 on mismatch.
- 2. **enroll**: check if UID already exists → 409 if duplicate → create card record with status="active" → 200.
- 3. **deactivate**: find card by UID → 404 if not found → set status="inactive", record timestamp → 200.
- 4. **info**: find card by UID → 404 if not found → return student name + current status → 200.

Response examples

action: "enroll" (200):

```
{ "allowed": true, "message": "ENROLLED", "sub": "Card saved" }
```

action: "deactivate" (200):

```
{ "allowed": true, "message": "DEACTIVATED", "sub": "Card disabled" }
```

action: "info" (200):

```
{ "allowed": true, "message": "FOUND", "sub": "Ahmed Hassan - Active" }
```

Error responses

Code	Condition	ESP32 Displays
401	Secret does not match ESP32_SECRET	SERVER ERROR
404	UID not found (deactivate / info)	UNKNOWN CARD / Not enrolled
409	UID already enrolled (enroll)	ALREADY / Registered

9 HTTP Status Code Reference

Code	Meaning	When to Use	ESP32 Display
200 OK	Success	Operation completed — use allowed:false for soft denial	Room message field
401 Unauth.	Bad secret	Secret field does not match ESP32_SECRET	SERVER ERROR
403 Forbidden	Gate rule failed	Room scan with no open entry session	ACCESS DENIED / Gate login first
404 Not Found	Unknown card	UID not in cards table	UNKNOWN CARD / Not enrolled
409 Conflict	Duplicate scan	Already on campus / already registered for session	ALREADY / Registered
422 Unprocess.	Bad payload	Missing required field or invalid UID format	SERVER ERROR
500 Server Err	Backend fault	Unhandled exception, DB connection failure	SERVER ERROR

Always return 403 (not 200 + allowed:false) when the gate rule is violated.
The ESP32 firmware checks the HTTP code to choose the display message — they are different.

10 Business Logic Rules

Rule 1 — Gate-First Enforcement

A student cannot register attendance for any lecture or lab without first scanning at the entry gate on the same calendar day. The check is: `campus_sessions WHERE student_id=X AND DATE(entry_time)=TODAY AND exit_time IS NULL`. If no such record exists, return HTTP 403.

Gate-first check implementation:

```
# Django ORM example
from datetime import date

open_session = CampusSession.objects.filter(
    student=student,
    entry_time__date=date.today(),
    exit_time__isnull=True
).first()

if not open_session:
    return JsonResponse({"error": "No entry session"}, status=403)
```

Rule 2 — Duplicate Scan Prevention

Every endpoint must check for duplicate scans within the same day/session before writing a new record. The ESP32 also enforces a 3-second client-side cooldown, but the backend must not rely on this. Use a unique constraint or an explicit query.

Duplicate prevention examples:

```
# Entry gate — already on campus check
if CampusSession.objects.filter(student=student,
    entry_time__date=date.today(),
    exit_time__isnull=True).exists():
    return JsonResponse({"error": "Already on campus"}, status=409)

# Room — already registered check
if RoomAttendance.objects.filter(student=student,
    room_id=room_id,
    scanned_at__date=date.today()).exists():
    return JsonResponse({"error": "Already registered"}, status=409)
```

Rule 3 — Card Status Check

Before processing any scan, verify the card's status is "active". Deactivated cards (lost/stolen) should be rejected with HTTP 403 and a "CARD INACTIVE" message — not 404, since the card exists but is blocked.

Card status enforcement:

```
# After looking up the card
if card.status != "active":
    return JsonResponse({
        "error": "Card deactivated",
        "allowed": False,
        "message": "CARD BLOCKED",
        "sub": "Contact admin"
    }, status=403)
```

Rule 4 — Rate Limiting

Rate-limit each endpoint to prevent replay attacks. Recommended minimum: 1 request per card UID per 3 seconds. This mirrors the ESP32 client-side cooldown and provides server-side protection if firmware is bypassed.

Rate limiting with cache:

```
# Using Django cache as a rate limiter
from django.core.cache import cache

cache_key = f"nfc_scan:{uid}:{endpoint_name}"
if cache.get(cache_key):
    return JsonResponse({"error": "Rate limited"}, status=429)
cache.set(cache_key, True, timeout=3) # 3-second window
```

11 Database Schema

The following schema covers the four tables needed by the NFC attendance system. It is designed to integrate with an existing university management database — foreign keys reference a students table that your Flutter app backend already maintains.

PostgreSQL DDL:

[illegible]

Recommended Indexes

Performance indexes:

```
-- Fast gate-check lookup (used on every room scan)
CREATE INDEX idx_sessions_student_date
ON campus_sessions (student_id, DATE(entry_time));
```

```
-- Fast duplicate check for room attendance
CREATE INDEX idx_room_att_student_room_date
ON room_attendance (student_id, room_id, DATE(scanned_at));

-- Card lookup by UID (used on every scan)
CREATE UNIQUE INDEX idx_cards_uid ON cards (uid);
```

cards.status Valid Values

Value	
"active"	Card is valid and operational
"inactive"	Deactivated by admin (lost, stolen, expired)
"pending"	Enrolled but not yet linked to a student account

12 Environment Configuration

All sensitive values and deployment-specific settings must be stored in environment variables. Never hardcode them in source files.

.env file:

```
# Required
ESP32_SECRET=your-strong-random-secret-minimum-32-chars
DATABASE_URL=postgresql://user:pass@localhost:5432/unitrack

# Optional
API_HOST=0.0.0.0
API_PORT=8000
SCAN_COOLDOWN_SECONDS=3 # server-side rate limit window per card
ROOM_CHECKIN_WINDOW_MINUTES=15 # how many minutes before/after lecture start
DEBUG=false
LOG_LEVEL=INFO
```

13 NVS Device Provisioning

Each physical ESP32 unit stores its configuration in NVS (Non-Volatile Storage) — a key-value store in the ESP32's flash memory that survives reboots and power cuts. These values must be set once during physical installation.

NVS Key	Type	Used In Mode	Description
mode	uint8	All	Active mode (0–3). Auto-saved on every touch cycle.
room_id	int32	MODE_ROOM	Room / lab ID from the database. Set once on installation. Default: -1 (w/ no room).
admin_action	string	MODE_ADMIN	Action string. Values: "enroll" "deactivate" "info". Default: "enroll".

Serial provisioning sketch:

```
// Paste into Serial Monitor once, then remove from code.
// Run after flashing firmware on a new unit.

#include <Preferences.h>
Preferences prefs;

void provisionDevice() {
  prefs.begin("nfc_sys", false);
```

```
// For a ROOM unit serving room ID 12:
prefs.putInt("room_id", 12);

// For an ADMIN unit set to enroll:
prefs.putString("admin_action", "enroll");

// For an ADMIN unit set to deactivate:
// prefs.putString("admin_action", "deactivate");

prefs.end();
Serial.println("Provisioning complete.");
}
```

Set `room_id` before installing a ROOM unit. If `room_id` is -1 (the default), the backend will receive `room_id`: -1 and cannot match it to a schedule.

14 Security Hardening Guide

Rotate ESP32_SECRET regularly

Change the shared secret every semester or after any suspected compromise. Update the `.env` on the server and reflash all ESP32 units with the new secret. Old units with the old secret will receive 401 and stop working — this is intentional.

Enable HTTPS / TLS

The current setup uses plain HTTP, safe only on a trusted LAN. Before connecting over the internet or campus WiFi, add TLS. The ESP32 supports it: use `http.begin(url, ca_cert_pem)` and point to your server certificate.

Validate UID format server-side

Always validate uid against the regex `^[0-9A-F]{2}:)*[0-9A-F]{2}$` before any database query. Reject malformed UIDs with 422. This prevents injection attacks.

Rate limiting per UID

Limit to 1 request per UID per 3 seconds at the server (mirrors the ESP32 cooldown). Use Redis or Django cache. Return 429 on violation.

Log all scan attempts

Write every scan — including 401, 403, 404, 409 — to `scan_audit` with timestamp, `device_id`, `uid`, `http_status`, and result. This audit trail is essential for investigating attendance disputes or security incidents.

Restrict API to LAN / VPN

If the backend is accessible from the internet, add IP allowlisting or place the NFC API behind a VPN so only campus network traffic can reach it.

Avoid logging UIDs in production

NFC card UIDs are personally identifiable. Avoid printing them to application logs at INFO level in production. Use DEBUG level or hash them.

15 Deployment Checklist

Backend Server

- Set ESP32_SECRET in .env (minimum 32 random characters)
- Set DATABASE_URL in .env
- Run database migrations — create all four tables + indexes
- Implement all four endpoints matching this specification
- Implement 403 gate-check logic in /attendance/room
- Implement card status check (active / inactive) in all scan endpoints
- Add rate limiting: 1 request / card / 3 s minimum
- Validate UID format (regex) on all endpoints
- Add scan_audit logging for all responses including denied
- Enable HTTPS before any production or internet-facing deployment
- Test all four endpoints with a REST client (Postman / cURL)

ESP32 Firmware

- Set WIFI_SSID and WIFI_PASSWORD
- Set API_BASE to server IP:port
- Set API_SECRET to match server ESP32_SECRET
- Set unique DEVICE_ID per physical unit (e.g. "gate-north")
- Flash firmware to all units
- Check Serial output: touch baseline should be above TOUCH_THRESHOLD

Device Provisioning (NVS)

- Provision room_id on all ROOM / LAB units
- Confirm room_id matches a real room ID in the database
- Provision admin_action on all ADMIN units ("enroll" or "deactivate")
- Verify each unit scans in the correct mode at boot

Hardware Installation

- Confirm PN532 SPI wiring: SCK=18, MISO=19, MOSI=23, SS=5
- Confirm OLED I2C address (0x3C or 0x3D)
- Confirm LED pins: Green=25, Red=26, Blue=27
- Confirm buzzer on GPIO 32, touch wire on GPIO 4 (T0)
- Check Serial output for [PN532] and [OLED] confirmation lines

End-to-End Testing

- Entry gate: scan enrolled card → 200 WELCOME → green LED
- Entry gate: scan same card again → 409 ALREADY → red LED
- Room: scan enrolled card (no prior entry) → 403 ACCESS DENIED → red LED

- Entry gate: scan card → room: scan same card → 200 REGISTERED → green LED
- Room: scan same card again → 409 ALREADY → red LED
- Exit gate: scan card → 200 GOODBYE → green LED
- Admin enroll: scan new card → 200 ENROLLED → blue LED
- All endpoints: send wrong secret → 401 → red LED
- All endpoints: scan un-enrolled card → 404 → red LED